Governance Control for Routed LLM Applications on Kubernetes: A Measured AKS Systems Study

Abstract

This article reports a measured AKS systems study of a Kubernetes-native governance control plane for routed LLM applications. The implementation combines a controller, Envoy Gateway, Envoy AI Gateway, Azure OpenAI and Azure AI Foundry backends, policy custom resources, and workload-level telemetry. On the current cluster, the system produces live FinOps attribution, executes quality-gate jobs with explicit accept, reject, and abstain outcomes, tracks workload-scoped budget state, emits declared-residency findings, actuates human-approved route changes, detects labeled gateway-bypass traffic from Tetragon-backed egress evidence, and enforces opt-in Ed25519-signed admission for both approval decisions and evidence ConfigMaps. The strongest supported claim is therefore functional: a Kubernetes control plane can reconcile several governance signals, persist auditable evidence on a live cluster, and reject unsigned governance mutations when signature enforcement is enabled. The follow-up campaigns refine the limits of that claim rather than eliminating them. On a quota-stable cluster-day, the completed E2 routing campaign shows that the routed baselines preserve objective exact-match and latency (overlapping 95% confidence intervals; 27 of 28 pairwise comparisons indistinguishable) while reducing normalized per-request cost by about 35%, and the E8 campaign separates near-noise data-plane latency from control-plane reconciliation overhead, which stays low and flat as the number of reconciled policy objects grows. The present study still does not establish universal routing superiority, broad inferential quality guarantees, full shadow-AI precision/recall, or production-scale performance.

1 Introduction

Networked LLM applications are increasingly deployed as Kubernetes workloads rather than isolated API clients. In that setting, the practical question is not only which model to call, but how to attach governance to each call at the

workload level: cost attribution, declared-residency policy, budget state, route-change safety, and detection of traffic that bypasses the approved gateway. This article studies a control-plane approach in which those concerns are represented as Kubernetes custom resources and reconciled against live gateway and egress telemetry [20, 18, 19, 41, 40].

The paper focuses on a systems question rather than on model-level benchmarking: when governance is expressed as Kubernetes resources and enforced through a live gateway stack, which governance signals can actually be observed, persisted, and acted upon on a real cluster, and which parts of the loop fail first under live provider and infrastructure constraints? On AKS we deployed the operator, the Envoy data plane, multiple demo applications, Azure OpenAI and Foundry-backed models, a lightweight Prometheus instance, and Tetragon [33, 24, 32, 34, 31, 45, 47]. That live deployment supports five measured capabilities:

- workload-level FinOps attribution from real gateway traffic,
- quality-gate execution with persisted evidence,
- budget tracking and phase progression,
- declared-residency reporting for non-compliant flows, and
- shadow-AI detection for direct LLM egress outside the gateway.

The revised version adds a sixth measured control-plane capability: opt-in signature-backed admission for governance-sensitive mutations. In the current AKS run, an unsigned `AIChangeRequest` approval patch was rejected at admission, the corresponding signed patch was accepted and actuated, and an opt-in evidence ConfigMap was likewise accepted when correctly signed and rejected after tampering. The main empirical contribution of the current version is therefore functional rather than comparative: live attribution, gate outcomes, budget states, route changes, rogue-egress findings, and signed-governance admission are all materialized as cluster objects or logs on AKS. The follow-up E2 and E8 campaigns are retained as bounded methodological evidence about the current evaluation environment, not as universal comparative or scalability results.

Several stronger claims remain out of scope. This paper does not establish routing superiority over external baselines, inferential quality guarantees with justified sample sizes, full privileged-adversary control-plane security, shadow-AI precision/recall, or production-scale performance. That narrower framing is consistent with FinOps governance guidance and with emerging AI risk-management expectations that emphasize traceability and policy controls in addition to raw model quality [10, 8, 9, 35, 42].

System family	Runtime route selection	Kubernetes workload attribution	Auditable control objects	Gateway-bypass visibility
RouteLLM / FrugalGPT	yes	no	no	no
AI gateways	partial	limited	limited	no
Gatekeeper / Kyverno	no	partial	yes	no
OpenCost	no	yes	limited	no
This work	yes	yes	yes	partial

Table 1: Positioning of the current system against adjacent families. "Partial" means the capability exists in a narrower form than the governance loop studied here.

2 Related Work

Existing systems in this problem space usually optimize one layer at a time: model routing, policy enforcement, gateway brokering, or cost attribution. Cost-quality routers such as RouteLLM and FrugalGPT focus on model-selection efficiency rather than Kubernetes-native governance workflows [36, 1]. More recent routing work continues this direction with difficulty-aware, test-time, and universal routers such as Hybrid LLM, BEST-Route, EcoRouter, and Universal Model Routing [3, 12, 50, 15]. These systems are the right conceptual baselines for routing quality/cost trade-offs, but they do not by themselves expose workload-scoped Kubernetes objects, approval workflows, or gateway-bypass evidence.

Gateway products such as Envoy AI Gateway, Kong AI Gateway, and Portkey concentrate on traffic brokering, fallback, observability, and provider-facing controls [4, 16, 43]. Kubernetes-native policy engines such as Gatekeeper and Kyverno are powerful for admission control and policy reporting, but they are not runtime LLM route-governance loops [37, 21, 22]. OpenCost provides vendor-neutral Kubernetes cost allocation, which is adjacent to but distinct from runtime governance-aware LLM routing [38, 39]. At the platform layer, the operator pattern, custom resources, and Gateway API provide the control-plane primitives used by this article's design [20, 18, 17, 19]. Gateway-bypass detection builds on eBPF-based egress observability, whose cloud-native deployment and portability are an active systems research area [47, 52].

The current paper therefore claims a narrower contribution than a full routing paper: a live Kubernetes control plane can reconcile multiple governance signals and persist workload-scoped evidence across routing, budget, quality, and egress paths. The paper is also distinct from evaluation-only frameworks such as HELM, MMLU, GSM8K, and MT-Bench, which inform quality assessment but are not themselves governance control loops [23, 13, 2, 51].

3 System Model and Problem Statement

We model each routed LLM interaction as a workload-attributed request emitted by a Kubernetes application. A request carries at least four governance-relevant attributes: namespace, application, provider/model target, and policy context. The control plane maintains this context through declarative resources:

- `AIProvider` and `AIModel` describe providers, models, zones, and cost priors.
- `AIFinOpsReport` aggregates observed usage into per-workload cost and token summaries.
- `AIQualityGate` evaluates source and candidate models against a golden dataset and records a gate verdict.
- `AIBudgetPolicy` tracks spend against a workload-scoped budget and surfaces a phase such as `WithinBudget` or `Warning`.
- `AISovereigntyPolicy` checks observed flows against declared zone rules and sensitive-data constraints.

The main problem addressed in the current draft is operational, not legal: can this control plane attach workload-scoped governance evidence to live AKS traffic, and can it do so for both approved gateway traffic and rogue traffic that bypasses the gateway? The adversary model considered in the current measurements is a non-malicious or locally misconfigured tenant workload, including a labeled rogue workload that calls an LLM endpoint directly. A malicious cluster administrator remains out of scope. The present study also treats the governance control plane itself as a potential weak point. In the current implementation, that weakness is no longer entirely hypothetical: the live AKS stack now enables opt-in Ed25519-signed admission for `AIChangeRequest` approval transitions and for evidence-bearing ConfigMaps, and the results section measures that unsigned or tampered writes are rejected. However, those controls remain narrow by design. They harden specific governance artifacts, not every mutable object in the cluster, and they do not by themselves establish full separation of duties or resistance against a privileged administrator who can alter webhook configuration, trusted keys, or the underlying nodes. Consequently, the paper evaluates a strengthened but still bounded integrity model rather than universal control-plane tamper resistance.

4 Design

The design centers on a closed-loop pattern, although the current measurements cover only part of that loop. Live application traffic is sent through Envoy AI Gateway with workload headers propagated by a lightweight sidecar proxy. The operator consumes the resulting OpenTelemetry-style Prometheus metrics and maps each request to the responsible namespace and application [4, 19, 45, 44, 41, 40]. Separate controllers then materialize governance views:

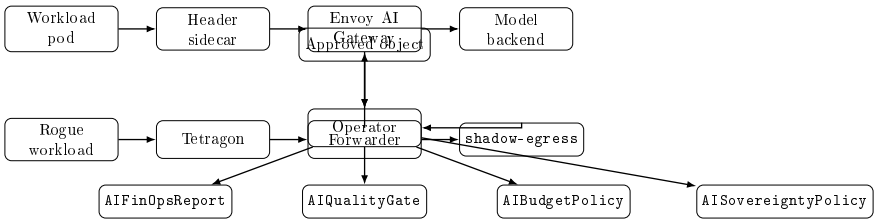

Figure 1: Architecture of the measured governance loop. The current paper evaluates all four paths, but with different depths of evidence.

- FinOps reconciliation aggregates tokens, request counts, model distribution, and cost estimates.
- Quality-gate reconciliation launches evaluation jobs and compares source and candidate observations.
- Budget reconciliation converts observed spend into phases relative to a per-application threshold.
- Sovereignty reconciliation inspects routed flows and, when available, shadow-egress observations derived from Tetragon.

The shadow-AI path is intentionally independent from the gateway. Tetragon observes kernel-level egress events, a forwarder converts those events into the `shadow-egress` ConfigMap, and the operator classifies direct calls to known LLM endpoints [47, 46]. This separation matters because a gateway-only system cannot observe traffic that never reaches the gateway.

5 Measured Quality and AIQualityGate

The implemented `AIQualityGate` is an auditable safety gate for route changes. It is not a learned black-box router and it is not a pure LLM judge. Instead, it combines measured evidence and operational telemetry into a composite score with explicit insufficiency behavior. This positioning is closer to governance-oriented evaluation than to benchmark-only reporting, even though benchmark suites such as HELM, MMLU, GSM8K, and MT-Bench motivate some of the signal choices [23, 13, 2, 51].

The default AI Quality Score is:

$$Q = 0.40 \cdot Correctness + 0.20 \cdot Reliability + 0.15 \cdot Latency + 0.15 \cdot Semantic + 0.10 \cdot Judged$$

with negative weights clamped to zero and the remaining weights normalized. When judge mode is disabled, the judged weight is removed before normalization. In this paper, however, the composite score is treated as an operator policy heuristic rather than as a validated scientific metric. The weights are implementation defaults, not values justified by sensitivity analysis or human-judgment calibration.

Correctness. The correctness dimension is computed from golden evidence. The implemented checks can combine normalized exact match, reference-vs-actual similarity, required-keyword presence, JSON validity, and field-level F1. This is an auditable deterministic evidence path, not a hidden online model score.

Reliability. Reliability is derived from live telemetry as:

$$100 \cdot \left(1 - \frac{errors + timeouts + invalidJSON}{requests}\right)$$

when request telemetry is available.

Latency. Latency uses a threshold-based score: latency at half the threshold maps to 100, latency at the threshold maps to 50, and latency at twice the threshold maps to 0, with linear interpolation between those points.

Semantic and judged signals. Semantic and judged scores are not fetched magically by the scoring engine. They are injected through the collected evidence. Judged score is therefore an auxiliary acceptability signal when enabled, not proof of objective correctness.

Insufficient-data behavior. If a weighted signal is missing, or if source/candidate evidence falls below `minSamples`, the gate returns `insufficient-data`. This is a key design property: the operator is designed not to approve a downgrade when the required evidence has not been observed.

Base decision rule. Let Q_s be the source-model score, Q_c the candidate-model score, and τ the configured `tolerancePoints`. The base decision is:

$$\texttt{candidate-safe} \iff Q_c \geq Q_s - \tau$$

Otherwise the verdict is `candidate-risk`. In the current code path, defaults are `minSamples=1` and `tolerancePoints=3` when not configured explicitly.

Statistical gate status. The controller code also contains an optional confidence-bound statistical path, but the present paper does not evaluate or rely on that path. The available AKS sample sizes are too small to support a statistically defensible inferential claim, so all result statements in this manuscript should be read as descriptive operational evidence rather than inferential quality proof.

Task-specific metrics. The current live AKS evidence exercises enterprise-style golden prompts plus two objective adapters in the external E2 campaign: GSM8K numeric exact match and MMLU multiple-choice exact match. Those objective signals are reported separately in the results because the composite

score itself has not yet been validated as a general-purpose quality metric. For this reason, the manuscript uses the gate to study accept/reject/abstain behavior and not to claim task-general objective correctness from one cluster run alone [13, 2].

The currently mounted golden datasets now contain four application-specific `ConfigMap` datasets (`finance`, `legal`, `marketing`, and `rh`) with 20 curated prompts each in the live E3 run. The repeated AKS campaign enforces `minSamples=20` and measures five repetitions per gate, which is enough to characterize stability versus instability for the tested applications, but still not enough to claim broad task-general calibration beyond this portfolio.

qualityeval evidence flow. The `qualityeval` job calls the production gateway endpoint for both the source and candidate models, serializes response evidence, and stores the material consumed by the gate. In Article 2, quality claims are therefore tied to real gateway-backed evaluation rather than offline fabricated comparisons.

Integration with routing and budget enforcement. The control loop couples actuation to quality evidence: a candidate model change is safe only when a matching quality gate reports a fresh candidate-safe verdict. For the declared-residency enforcement path this coupling is implemented — an enforce-mode reroute consults the latest matching `AIQualityGate` and, on a non-candidate-safe or missing verdict, withholds the route change and opens a `Pending AIChangeRequest` for human approval instead of actuating automatically. Budget-driven fallback does not yet apply the same gate, so the results section separates what the gate measured from what each controller path actually enforced.

5.1 Justification of the composite weights

The composite quality score is a weighted sum of measured dimensions, and the default weights (correctness 0.40, reliability 0.20, latency 0.15, semantic 0.15, judge 0.10) are product defaults rather than values derived from data. Fixing such weights a priori is a documented weakness: recent work shows that aggregating multi-dimensional evaluation is volatile and sensitive to scoring choices [6], that judge-based dimensions are themselves sensitive to design decisions [49], and that seemingly reasonable dimensions can be task-dependent or even negatively correlated with reference quality unless they are audited and re-normalized [48]. We therefore do not defend a single weight vector; instead we report how the measured results depend on the weights.

Separation of answer quality from operational health. Re-analyzing the measured per-dimension scores of the E3 run shows that the two operational dimensions (reliability and latency) are *saturated at* 100 *for every gate*, because all tested models answered with a zero error rate and well below the

Gate	Answer-quality	Operational	Composite
finance-risk-assistant	56.2	100.0	73.2
legal-contract	44.4	100.0	66.0
marketing-content	57.2	100.0	73.8
rh-chatbot	50.1	100.0	69.5

Table 2: Measured E3 sub-scores (mean over five repetitions). Operational dimensions are saturated, so the composite is the answer-quality sub-score plus a constant operational offset. Evidence: `E3_quality_gate/aks-live-20260708T215509Z`.

latency threshold. Under the default weights these two dimensions therefore add a constant offset of ≈ 38.9 points to every composite, independently of answer quality. The effect is visible in Table 2: the `legal` gate has a measured answer-quality sub-score of 44.4 (correctness and semantic only) but a reported composite of 66.0. To avoid presenting operational availability as answer quality, we report the answer-quality sub-score (correctness and semantic, re-normalized) separately from the operational sub-score.

Weight sensitivity. We recompute the composite from the same measured dimension scores under five weighting schemes (the default, an answer-only scheme that zeroes the operational dimensions, a correctness-heavy scheme, equal weights, and a scheme that halves the operational weights). The absolute composite is weight-sensitive — the `legal` gate ranges from 44.4 to 73.4 across schemes — but the *ordering of gates by quality is identical under every scheme* (marketing > finance > rh > legal). The comparative conclusion of the gate — which application's candidate is relatively safer or riskier — is therefore invariant to the weight choice within this run, even though the headline number is not. We consequently rely on the answer-quality sub-score and the cross-scheme ordering rather than on the absolute composite value, and we treat data-driven weight calibration against objective ground truth [48] as the principled next step, noting that the current open-ended golden datasets do not carry the objective labels such calibration requires.

6 Implementation

The current implementation was exercised on a real AKS cluster rebuilt on 2026-07-08. The live stack follows the Kubernetes operator pattern on AKS and uses Azure identity and observability building blocks that are standard in Microsoft guidance [20, 18, 33, 24, 30, 28, 27]. The live stack includes:

- the operator image `ghcr.io/ihsenalaya/ai-sovereign-finops-operator/controller:0.5.12`,
- Envoy Gateway and Envoy AI Gateway,

- Azure OpenAI France and US deployments,
- an Azure AI Foundry endpoint used for the EU-backed Mistral path,
- an initial four-application measured subset (`rh/chatbot-rh`, `finance/risk-assistant`, `legal/contract-review`, and `marketing/content-writer`),
- a later portfolio extension to 13 normal gateway-routed applications and 2 rogue applications, with labeled deployments recorded on AKS,
- a lightweight in-cluster Prometheus deployment for operator scraping, and
- Tetragon with an egress tracing policy plus a labeled rogue workload.

Reproducible run folders were generated for FinOps attribution, quality gates, budget tracking, declared-residency reporting, workflow state capture, and shadow-AI detection. The model-serving side combines Azure OpenAI access patterns with Azure AI Foundry-hosted endpoints and observability surfaces [32, 34, 25, 31, 26, 29].

7 Methodology

This paper reports measured AKS behavior from one rebuilt cluster on 2026-07-08. The reported runs are artifact-driven and exploratory rather than statistically complete, and all headline claims are restricted accordingly.

The live methodology used here is straightforward:

- deploy the operator, gateway, providers, and applications on AKS;
- generate real application traffic through the gateway from workload pods;
- collect operator reports, gateway metrics, quality-gate ConfigMaps, budget policy status, and declared-residency findings;
- deploy Tetragon and a labeled rogue workload that calls `api.openai.com` directly;
- convert observed egress events into `shadow-egress` and verify that the operator emits a shadow-AI event.

Instrumentation is grounded in standard telemetry stacks rather than bespoke tracing logic: Prometheus captures gateway-facing metrics, OpenTelemetry conventions inform signal naming, and Tetragon provides kernel-level egress evidence outside the gateway path [45, 44, 41, 40, 47]. Cost interpretation is aligned with OpenCost-style cluster attribution concepts and FinOps governance terminology, while the AI provider layer is supplied by Azure OpenAI and Azure AI Foundry [38, 39, 8, 7, 25, 31].

The rebuilt AKS environment now includes 13 normal gateway-routed applications across multiple namespaces and 2 rogue applications. Only a subset

has deep measured evidence, and the manuscript distinguishes clearly between deployed coverage and experimentally exercised coverage.

Two additional campaigns were executed specifically to reduce the earlier gaps. E2 issued 1,536 live gateway requests across eight routing baselines and three quota-aware windows, with 60 objective items and 4 open-ended items per baseline in each window. E8 deployed workload profiles of 12, 50, and 100 applications on the same AKS cluster, measured both direct-path and gateway-path latency from workload pods, and recorded the point at which the cluster stopped making additional deployments available. The current run set is sufficient for descriptive systems reporting with paired confidence intervals on the routing study, but not for multi-cluster generalization, robust false-positive/false-negative estimation, or production-scale capacity claims. This is especially important because routing baselines in the literature are usually evaluated on benchmark suites or offline traces rather than on a live governed gateway stack [36, 1, 3, 12, 50, 15].

The artifact surface is explicit. The paper-level evidence is backed by versioned manifests, run folders, and replay scripts under `article2/experiments/`, `article2/scripts/`, and `article2/reports/`. In particular, E2 and E8 are backed by `run_e2_routing_campaign.py` and `run_e8_performance_campaign.sh`, while quality-gate and control-path evidence is captured in the dated AKS run folders and the claims-to-evidence indexes.

8 Results

The current AKS deployment demonstrates that the governance control plane is operational on live Azure infrastructure. The strongest observations from the latest run are descriptive and directly traceable to Kubernetes resources and captured logs.

FinOps attribution. The live `AIFinOpsReport` for `ai-report-all` attributed a total consumption of 1,808 input and 13,056 output tokens across nine (namespace, application, model) triplets. The dominant consumer was `finance/risk-assistant` on `gpt-us-mini` (a non-compliant US provider), accounting for 1,299 input and 10,882 output tokens. To move E1 beyond a controller smoke test, we independently reconstructed the per-triplet token attribution from the raw Envoy AI Gateway token telemetry in `metrics/gateway_metrics.prom` and compared it to the report: the per-triplet input and output token counts matched exactly, since both are derived from the same gateway token histogram, and the top-consumer ranking was reproduced exactly (`gpt-us-mini` first, then `mistral-large-latest`, then `gpt-france-mini`).

We further audited the request counter, which a reviewer could suspect of attributing consumption to error responses. Decomposing the gateway duration histogram by the `error_type` label shows that for `finance/risk-assistant` on `gpt-us-mini` the gateway recorded 85 successful responses and 660 error responses (all `error_type="_OTHER"`, `response_model="unknown"`, consistent

ID	Strongest current result	Status
E1	Live `AIFinOpsReport` plus independent raw-telemetry cost re-analysis on AKS	Partial
E3	Repeated five-run quality-gate campaign with 20-prompt golden datasets, quantified stability, and two `insufficient-data` controls	Partial
E4	Live budget phases observed through `Warning`, `Critical`, and `Exceeded`	Partial
E5	Report-only findings, enforce-mode FR-only reroute with rollback, and evidence-gated enforcement that withholds an un-evidenced reroute and escalates it to a `Pending AIChangeRequest`	Supported
E6	Live `AIRouteOverride`, approved actuation, and rejected no-op `AIChangeRequest` behavior	Partial
E7	Tetragon-backed gateway-bypass detection with two true-positive operator warning paths	Partial
E2	Eight-baseline live routing campaign completed across three windows with zero HTTP errors and pairwise statistics	Partial
E8	Data-plane latency profiles (12/50/100 apps) near 90–110 ms plus a controller-overhead campaign: reconcile latency stays flat and the workqueue stays empty as reconciled objects scale	Partial
T5	Live Ed25519-signed admission rejected an unsigned approval patch and a tampered evidence ConfigMap on AKS	Supported

Table 3: Summary of the current Article 2 experiment status on AKS.

with upstream 429 throttling), while every other measured triplet had zero errors. Crucially, the token-usage metric is emitted by the gateway *only for successful responses*: the 660 error responses consumed *zero* input and output tokens, and all of this triplet's 1,299 input and 10,882 output tokens were produced by the 85 successful calls. Consumption attribution is therefore unaffected by the errors. What was misleading was the `requests` field itself: the earlier collector took the maximum count across all duration series, including the error series, so the "660 requests" reported for this triplet in earlier versions was in fact an *error-dominated count* (660 failed attempts under throttling), not 660 completed calls. We redefine `requests` to count successful, token-producing calls only (85 for this triplet) and expose the error count separately, and we freeze this semantics with a unit test on the collector so request and token accounting can no longer diverge.

A later post-extension portfolio snapshot already shows that attribution is not limited to the original four-app subset. In that later report, new portfolio applications such as `finance/invoice-extractor` and `legal/compliance-classifier` appeared as separate application entries with their own request counts, costs,

Gate	Target	Stability	Verdict pattern	Mean	Std	N	Reps
finance-risk-assistant-quality	finance/risk-assistant	stable-safe	5 safe / 0 risk	73.219	1.017	20	5
legal-contract-quality	legal/contract-review	unstable	4 safe / 1 risk	66.013	1.534	20	5
marketing-content-quality	marketing/content-writer	stable-safe	5 safe / 0 risk	73.832	0.418	20	5
rh-chatbot-quality	rh/chatbot-rh	stable-risk	0 safe / 5 risk	69.495	0.345	20	5

Table 4: Detailed AKS `AIQualityGate` outcomes from the latest E3 campaign.

and routing-score rows. This is useful as an implementation milestone because it demonstrates that the gateway attribution path scales beyond the earliest smoke-test tenants, even though the full prompt-level routing baseline study remains incomplete.

Quality gates. The updated 2026-07-08 AKS quality campaign replaced the earlier one-prompt run with four application-specific golden datasets of 20 curated prompts each, `minSamples=20`, and five repetitions per gate. The resulting evidence is no longer a single-shot controller demonstration; it quantifies stability under live response variance. Two gates were stable-safe across all five repetitions: `finance-risk-assistant-quality` averaged 73.219 with standard deviation 1.017, and `marketing-content-quality` averaged 73.832 with standard deviation 0.418. One gate was stable-risk across all five repetitions: `rh-chatbot-quality` averaged 69.495 with standard deviation 0.345 and never crossed the tolerance boundary. The most informative case was `legal-contract-quality`: it produced one `candidate-risk` verdict followed by four `candidate-safe` verdicts, yielding an unstable stability label with mean 66.013 and standard deviation 1.534. We also retained two auxiliary gates: one without `evidenceRef`, which remained `insufficient-data`, and one with a non-compliant candidate lacking both evidence and telemetry, which also remained `insufficient-data`. The robust conclusion is therefore stronger than simple accept/reject coverage: the live AKS run now demonstrates reproducible safe gates, reproducible risky gates, quantitatively unstable gates, and explicit abstention paths on real evidence.

Application coverage. The deployed portfolio now contains 13 normal applications and two rogue workloads, but only a measured subset has been exercised deeply enough for the strongest result statements. The generated application matrix makes that boundary explicit: `finance/risk-assistant`, `legal/contract-review`, and `marketing/content-writer` are already tied to E1/E2-ready workload definitions, while most extension applications are cur-

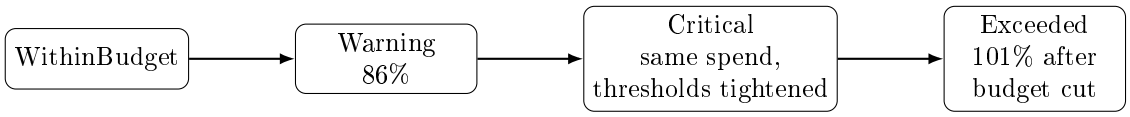

Figure 2: Observed budget-policy phase progression for `finance-budget`. The latter two transitions were induced by threshold and budget changes, not by additional spend.

rently only E3-ready from a policy and dataset perspective. This distinction matters because it prevents us from overstating multi-tenant evaluation breadth.

Budget tracking. The most informative budget result is `finance-budget`. Under real traffic it entered the `Warning` phase at 86% of its configured monthly budget. We then captured a controlled phase progression on the live cluster by tightening the policy thresholds: with `criticalThresholdPercent=80` and `hardLimitPercent=95`, the same observed consumption moved the policy to `Critical`; after temporarily lowering the configured budget below the observed consumption, the policy moved to `Exceeded` at 101%. The operator emitted matching `BudgetThreshold` warning events for both transitions. The other three application budgets remained within budget. This supports live phase-transition behavior, but not automatic fallback actuation, because no cheaper observed managed fallback model exists in the current catalog.

Human workflow and reroute actuation. In the earlier workflow run on operator image `ghcr.io/ihsenalaya/ai-sovereign-finops-operator/controller:0.5.12`, a live `AIRouteOverride` actuated a reroute from `gpt-us-mini` to `gpt-france-mini`. The corresponding `AIGatewayRoute` changed backend from `greenops-openai-us` to `greenops-openai-fr` and added a `bodyMutation` that rewrote the request model field. Deleting the override restored the original route. We then exercised `AIChangeRequest`: in `Pending`, the route remained unchanged; after patching `spec.approval=Approved`, the request became `Actuated` and applied the same reroute. In a separate rejected-path run, the request stayed non-actuating while `Pending` and the route still remained unchanged after `spec.approval=Rejected`. A newer run on image `0.5.17` then re-measured the automatic recommendation intake path: a live `AIRoutingPolicy` evaluated six applications, emitted four recommendations, and created four labeled pending `AIChangeRequest` objects on AKS, including a `gpt-us-mini` $\rightarrow$ `gpt-france-mini` request for `risk-assistant`; while those generated requests remained `Pending`, the route stayed unchanged. This demonstrates live human approval gating, explicit rejection behavior, route actuation after approval, and live `AIRoutingPolicy` handoff into approval-gated change requests, but not automatic rollback on deletion, which is not implemented in the current controller.

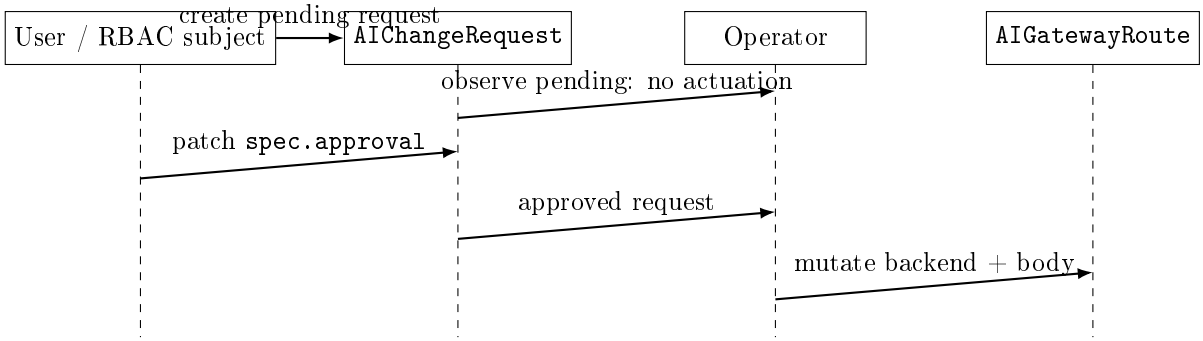

Figure 3: Measured `AIChangeRequest` path. In the latest AKS run, the same path was subsequently hardened with opt-in Ed25519-signed admission: the unsigned approval patch was denied, while the signed one was accepted and actuated.

T5 signed admission. The latest AKS campaign extended the human-workflow path into a control-plane integrity result. After upgrading the live operator to image `ghcr.io/ihsenalaya/ai-sovereign-finops-operator/controller:0.5.17` and enabling trusted Ed25519 public keys, an unsigned patch attempting to set `spec.approval=Approved` on `finance-reroute-signed` was rejected at admission. The same request was then signed over the canonical tuple `(namespace, name, action, sourceModel, targetModel, approval)` and accepted, after which the request reached `Actuated` and rerouted `greenops-openai-us` to `greenops-openai-fr`. The same AKS run also enabled a second validating webhook for opt-in evidence ConfigMaps. A correctly signed `shadow-egress-signed-proof` ConfigMap was admitted, while a later update that changed its payload without recomputing the signature was rejected. This does not prove whole-cluster tamper resistance, but it does convert a former caveat into a measured property: two governance-critical mutation paths are now cryptographically bound to trusted keys when the feature is enabled.

Declared-residency reporting. The current `AISovereigntyPolicy` produced both report-only and enforce-mode evidence. In `reportOnly`, the operator emitted a critical finding and a warning for `finance/risk-assistant` traffic sent to `openai-us` in the forbidden `US` zone. We then enabled `enforce`. With `allowedZones={FR,EU}`, the controller first selected the cheapest compliant target (`mistral-small`) but did not mutate the route because that target was not routable in the live gateway stack. After temporarily narrowing `allowedZones` to `FR`, the cheapest compliant routable target became `gpt-france-mini`, and the operator actuated a real reroute of `greenops-openai-us` to `greenops-openai-fr` with an `enforced-reroutes` annotation and request-body model rewrite. Restoring `reportOnly` restored the original route automatically.

Evidence-gated enforcement. Earlier versions noted that this residency reroute changed the served model without first consulting the quality gate that the design recommends for route changes. We closed that gap: with `requireQualityEvidenceForReroute` enabled, the enforce path now consults the latest matching `AIQualityGate` (same source and candidate models) before mutating a route, and actuates only on a fresh candidate-safe verdict; a candidate-risk, insufficient-data, or missing verdict blocks automatic actuation and instead opens a `Pending` `AIChangeRequest` for human approval, reusing the routing-to-approval handoff. On the live cluster we exercised the escalation path directly: with no candidate-safe gate for the `gpt-us-mini` → `gpt-france-mini` pair, the operator withheld the reroute (the `AIGatewayRoute` backend was left unchanged) and created the escalation request `sov-regulated-france-policy-gpt-us-mini` in `Pending`. The complementary actuation branch — a fresh candidate-safe verdict authorizing the reroute — is covered by a deterministic unit test of the pure decision function, since minting a genuine candidate-safe verdict on the cluster requires a full golden-evidence evaluation of that specific model pair. The safety invariant "no route change without adequate evidence" is therefore now enforced for the residency-enforcement path rather than merely recommended.

E2 routing baselines. The repaired live E2 campaign completed on AKS after adding quota-aware pacing and interleaving requests across the three provider backends. The final run issued 1,536 requests through the live gateway across eight routing baselines, three windows, 60 objective items per baseline, and 4 open-ended items per baseline, with zero HTTP errors in every window. The strongest result is therefore no longer "the experiment was blocked," but that the governed live stack can sustain a repeated-window routing comparison under managed quotas without upstream failure contamination. Each baseline was evaluated on $N = 180$ objective items (60 items over three windows). Objective exact-match ranged from 0.522 for `B8_routing_score_gate_change_request` to 0.583 for both `B3_static_namespace` and `B4_policy_only_premium`, but the Wilson 95% confidence intervals overlap for every baseline (Fig. 4). Pairwise McNemar tests paired by item and Holm-corrected across all 28 baseline pairs find a single significant difference — `B4_policy_only_premium` versus `B8_routing_score_gate_change_request` (Holm-adjusted $p = 0.027$, the two extremes) — while the other 27 pairs are statistically indistinguishable. Token consumption per request was essentially identical across all baselines (about 94 tokens per request), so the baselines perform equivalent work and differ only in model selection; expressed as normalized cost per request, the routed baselines (`B2/B5/B6/B7/B8`) were about 35% lower than the premium-like baselines (`B1/B3/B4`), a separation driven entirely by per-token model price at equal token volume rather than by any difference in tokens consumed. Mean per-request latency ranged only from 134 ms (`B7`) to 175 ms (`B3`) with overlapping 95% confidence intervals, so no baseline latency ordering is statistically supported either. The defensible E2 result is therefore an *equivalence* on objective quality and latency across baselines — with the single noted extreme-pair exception —

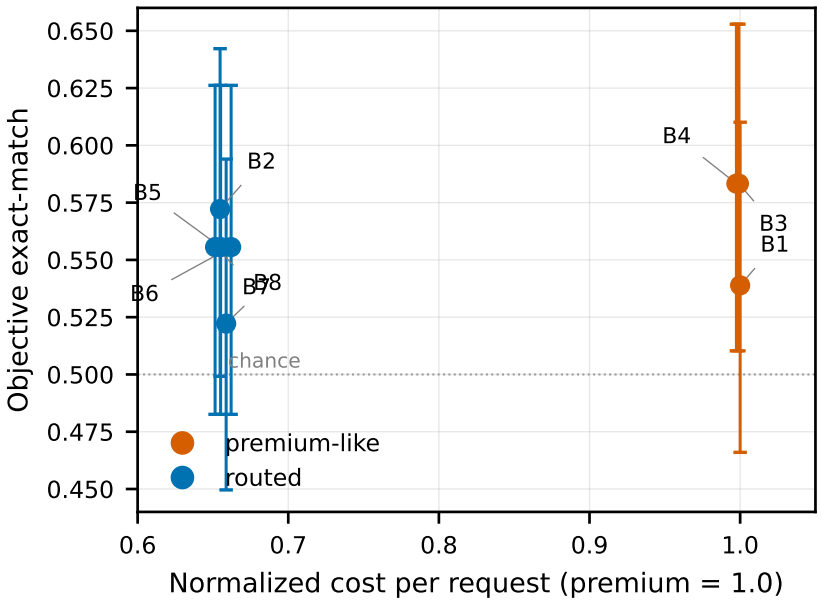

Figure 4: Measured E2 cost/quality trade-off. Horizontal axis: normalized cost per request (premium-like baseline = 1.0, dimensionless). Vertical axis: objective exact-match with Wilson 95% confidence intervals ($N = 180$ each). The routed baselines (blue) cluster near 0.65 normalized cost at exact-match statistically indistinguishable from the premium-like baselines (orange) near 1.0.

combined with a robust and consistent normalized-cost reduction for the routed baselines. This is a cleaner statement than the earlier "trade-off": on this workload the routed policies preserve objective quality and latency while materially lowering cost. The open-ended dimension is reported separately and only descriptively, since it rests on 12 items per baseline and has limited discriminative power.

E8 performance and scale. The updated E8 campaign replaced provider-dependent latency with a controlled mock backend and measured both direct-path and gateway-path latency while increasing the number of deployed applications. All three workload profiles completed their measurement steps with zero request errors. At 12 applications, all 12 pods were ready and direct/gateway p50 latencies were 89.1/90.2 ms. At 50 applications, 25 pods were ready and 25 remained pending, while direct/gateway p50 latencies were 92.2/90.2 ms over 125 requests per path. At 100 applications, 25 pods were ready and 75 remained pending, while direct/gateway p50 latencies were 90.1/89.2 ms over 125 requests per path. The central result is therefore not a precise positive gateway-overhead estimate, because the observed direct-versus-gateway differences are small and sometimes change sign. Instead, the defensible conclusion

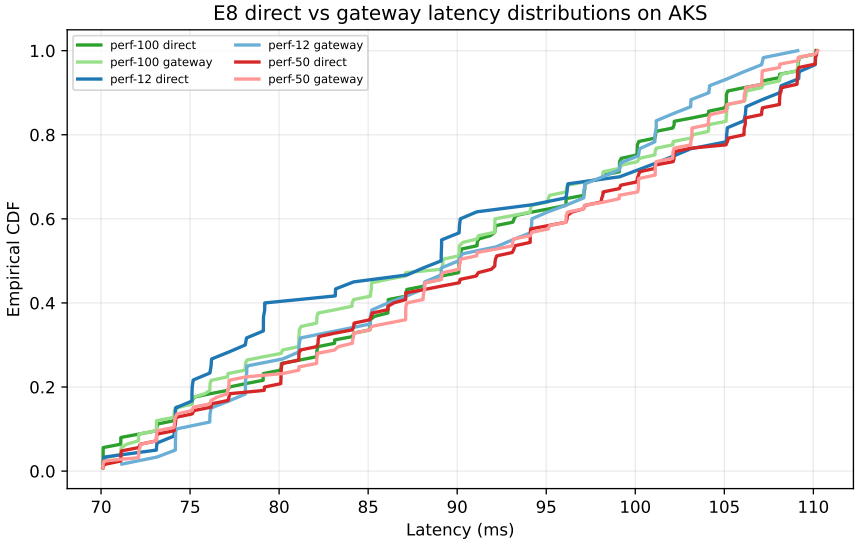

Figure 5: Measured E8 empirical latency distributions for direct and gateway paths across the 12-, 50-, and 100-application profiles. The dominant scaling limit in this run was cluster capacity, not a large deterministic gateway penalty.

is that gateway-path latency remained within measurement noise of the direct path across the tested profiles, while the cluster itself stopped increasing delivered capacity after roughly 25 ready applications. This is a stronger systems result than the earlier artifact because it separates two phenomena that were previously entangled: data-plane latency stayed near 90–110 ms once workloads were running, whereas the dominant bottleneck was scheduling capacity on the current AKS quota and node-pool shape. The claim should therefore remain explicitly descriptive and cluster-specific: E8 now documents a controlled latency comparison plus an observed capacity ceiling on the tested cluster, not a production-scale scalability law for the operator.

Controller overhead. E8 characterizes the data plane, but the contribution of this paper is the control plane, so we additionally measured controller overhead directly from the operator's controller-runtime metrics while growing the number of reconciled policy objects (a mock-backend campaign with no LLM calls). Across three tiers of 40, 120, and 240 reconciled objects (equal numbers of `AIBudgetPolicy` and `AISovereigntyPolicy`), the operator processed 172, 555, and 1,260 reconciles per measurement window respectively, yet reconcile latency did not degrade: p50 stayed at 21.7/20.6/19.8 ms and p95 actually fell from 91.3 to 66.2 ms as load increased (Fig. 6), the workqueue depth was zero at every scrape (no backlog), and resource use stayed low and flat — CPU rose only from 0.012 to 0.020 cores and resident memory remained near 53 MiB (Fig. 7).

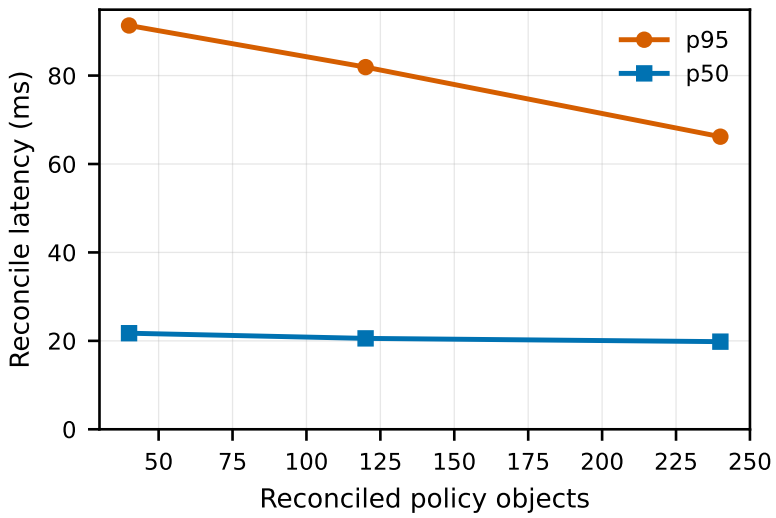

Figure 6: Controller reconcile latency (p50/p95) versus the number of reconciled policy objects, live AKS controller-runtime metrics. Latency stays flat as load grows.

The induced API/etcd write rate from the ConfigMap-as-bus and status-update pattern grew modestly from 2.3 to 5.5 writes/s. The defensible control-plane result is therefore that reconciliation overhead is small and does not grow with the number of governed objects over the tested range; the earlier "capacity ceiling" was a node-pool pod-density limit of the data plane, not a controller limit, and is discussed as a cluster-shape validity threat in Section 10 rather than as a scalability law.

Shadow-AI detection. The updated shadow-AI campaign moved beyond true-positive-only evidence and produced a small live FP/FN matrix on AKS. Direct rogue calls to both `api.openai.com` and `api.anthropic.com` were detected end-to-end: Tetragon `tcp_connect` observations were converted into `shadow-egress` entries, and the operator emitted `ShadowAI` warnings for `shadow-t4/tp-openai` and `shadow-t4/tp-anthropic`. A direct-IP OpenAI variant was also detected, showing that the current bridge can still classify some IP-only calls by mapping destination IPs back to known LLM hosts. However, the same campaign also exposed concrete weaknesses. Two near-miss Anthropic workloads, `anthropic.com` and `docs.anthropic.com`, were falsely classified as `api.anthropic.com`, yielding two false positives among four negative scenarios. Two positive scenarios were missed: a proxy-mediated OpenAI call did not produce a detection for the client workload, and a call to an unlisted endpoint (`api.perplexity.ai`) produced no detection at all. Across the nine scenarios this is 2 false positives out of 4 negative scenarios and 2 false negatives out of 5 positive scenarios; we

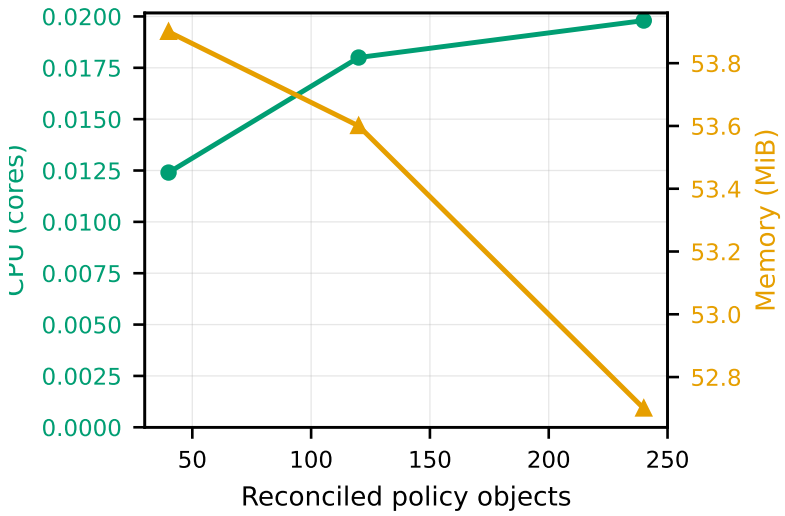

Figure 7: Operator CPU (cores) and resident memory (MiB) versus reconciled-object count. Both remain low and nearly flat.

report these raw counts rather than rates, since two-decimal rates over so few scenarios would imply a precision the sample does not support. This is not a polished detection result, but it is a publishable systems result: the current implementation works for direct known-host egress, partially survives direct-IP use, and fails in predictable ways under near-miss hostnames, proxy indirection, and unlisted endpoints because endpoint classification in this AKS run depended on Tetragon `tcp_connect` events plus destination-IP matching against a fixed known-host list rather than on TLS SNI or a broader endpoint catalog.

Scope note. The limits of E2, E3, E7, and E8 are consolidated in Section 10 rather than repeated here.

9 Discussion

The current evidence suggests that the most immediately useful aspect of the system is not automatic route optimization but unified visibility. Even without turning on enforcement modes, a platform team can already observe who spent money, which application violated a declared zone policy, which candidates failed a quality gate, and whether an application bypassed the gateway entirely. That emphasis on observability and governed change aligns more closely with FinOps and AI risk-management practice than with pure benchmark chasing [10, 8, 9, 35, 14].

The completed E2 campaign sharpens that point. Once stale backends and stale API-key secrets were repaired, the limiting factor was no longer the con-

troller path but upstream provider throttling. For this paper, that observation is best read as a protocol warning about live managed-endpoint evaluation rather than as a comparative routing result.

The completed E8 campaign reveals a different weakness. On the current cluster shape, the experiment became scheduling-capacity bound before there was evidence of controller saturation. The central lesson is therefore not that the controller added a large deterministic overhead, but that cluster shape and pod-density ceilings can dominate the measured envelope long before the reconciliation path itself becomes the bottleneck.

A design goal is that governance actuations should be coupled to adequate quality evidence. For the residency-enforcement path this coupling is now enforced: when evidence checking is enabled, a reroute is actuated only on a fresh candidate-safe verdict from a matching quality gate, and otherwise it is withheld and escalated to a human-approved change request (Section 8). We demonstrated the escalation branch live and cover the actuation branch with a unit test. The coupling is not yet applied on every actuation path — budget-driven fallback, in particular, still acts on spend phase alone — so the invariant holds for residency enforcement but is not yet a global property of the control plane; extending it uniformly is future work.

The shadow-AI follow-up also exposed a practical systems lesson. The eBPF signal itself was present for both OpenAI and Anthropic rogue traffic, but the evidentiary bridge into the operator depended on the `shadow-egress` handoff remaining populated from current Tetragon exports. In other words, the hard problem was not endpoint recognition but operationalizing the bridge as a continuously refreshed telemetry path.

The manuscript also has to be explicit about control-plane security limits. The latest implementation no longer leaves the approval and evidence paths entirely unprotected: when trusted keys are configured, `AIChangeRequest` decisions and opt-in evidence ConfigMaps are checked by validating webhooks, and the AKS results show that unsigned or tampered writes are denied. That is a real strengthening of the control plane and gives the paper a clearer identity than a pure future-work note. At the same time, the safe interpretation remains bounded. The measured property is admission-time integrity for two governance-sensitive object classes, not universal tamper evidence under privilege compromise, because a sufficiently privileged cluster administrator could still alter webhook configuration, trusted keys, or the underlying nodes. The paper should therefore claim measured signed-governance admission, not complete control-plane immutability.

10 Validity and Scope

This study is intentionally narrow. It supports live AKS claims about FinOps attribution, quality-gate execution, budget tracking through `Warning`, `Critical`, and `Exceeded`, declared-residency reporting plus an FR-only enforce-mode reroute, human approval gating with route actuation, a live shadow-AI matrix that

includes both true positives and measured failure modes, a zero-error eight-baseline routing campaign executed across three quota-aware windows, a controlled three-profile latency-and-capacity study that separates gateway-path behavior from cluster scheduling saturation, and opt-in Ed25519-signed admission for approval decisions and evidence ConfigMaps. The compliance framing is deliberately limited to configured policy enforcement and declared-residency observability rather than formal legal certification, which is important when reading the work against GDPR, the EU AI Act, NIST AI RMF, OWASP guidance, and ISO/IEC 42001 [11, 5, 35, 42, 14]. It does not yet support:

- universal routing-superiority claims across all baselines and workloads,
- strong inferential headline claims beyond this one live cluster-day and provider quota regime,
- broad residency-enforcement claims that ignore live catalog routability limits and quality-gate bypasses,
- whole-cluster tamper-evidence or privileged-adversary audit-integrity claims,
- full shadow-AI precision/recall characterization or robust adversarial-evasion coverage, or
- production-scale performance and scalability claims.

Additional validity threats are specific and concrete: all evidence comes from one cluster rebuilt on one day; E2 is now quota-stable but still reflects one live provider catalog and one managed-quota configuration rather than a cross-region or cross-provider routing benchmark, and its open-ended component is small relative to the objective component; E8 shows the capacity ceiling of the current AKS quota and node-pool shape, not an architectural upper bound for the operator, and its direct-versus-gateway latency differences should be interpreted as near-noise rather than as a precise positive overhead constant; the improved E3 campaign still covers only four application-specific datasets despite raising them to 20 prompts and five repetitions, and one gate (`legal-contract-quality`) remained verdict-unstable across repetitions; the new shadow-AI matrix is still small and depends on IP-to-host classification for known endpoints, which explains both the observed false positives on Anthropic near-miss domains and the false negatives on proxy-mediated and unlisted endpoints; and the T5 signed-admission result applies only to the two measured object classes (`AIChangeRequest` and opt-in evidence ConfigMaps), not to arbitrary cluster objects. Accordingly, the paper should be read as a measured AKS systems study with a deliberately narrowed claim set, not as a final comparative routing, security, or scalability paper.

11 Conclusion

The rebuilt AKS deployment now supports a clearer systems paper than the earlier partial draft. Real workloads, real Azure model endpoints, and real

operator reconciliations demonstrate live FinOps attribution, quality-gate execution, budget-state progression, declared-residency findings, route actuation gated by quality evidence with human escalation, and shadow-AI detection. The central contribution of the present version is this functional governance loop on a live cluster; the remaining comparative, statistical, and scalability limits are explicitly consolidated in Section 10.

References

[1] Lingjiao Chen, Matei Zaharia, and James Zou. Frugalgpt: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[2] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[3] Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid llm: Cost-efficient and quality-aware query routing. In *The Twelfth International Conference on Learning Representations*, 2024.

[4] Envoy AI Gateway Authors. Envoy ai gateway documentation. Project Documentation, 2026.

[5] EUR-Lex. Rules for trustworthy artificial intelligence in the EU. Legal Summary, 2026.

[6] Benjamin Feuer, Chiung-Yi Tseng, Astitwa Sarthak Lathe, Oussama Elachqar, and John P. Dickerson. When judgment becomes noise: How design failures in LLM judge benchmarks silently undermine validity, 2025.

[7] FinOps Foundation. Data ingestion. FinOps Framework Capability, 2026.

[8] FinOps Foundation. Finops framework. Framework Documentation, 2026.

[9] FinOps Foundation. Governance, policy & risk. FinOps Framework Capability, 2026.

[10] FinOps Foundation. What is finops? FinOps Framework, 2026.

[11] GDPR-Info.eu. General data protection regulation (GDPR). Legal Text Portal, 2026.

[12] Kaixin Go, Uri Gadot, Navdeep Kumar, Kfir Yehuda Levy, and Shie Mannor. Best-route: Adaptive llm routing with test-time optimal compute. In *Proceedings of the 42nd International Conference on Machine Learning*, 2025.

[13] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.

[14] International Organization for Standardization. Iso/iec 42001 artificial intelligence — management system. Standard Overview, 2023.

[15] Wittawat Jitkrittum, Harikrishna Narasimhan, Ankit Singh Rawat, Jeevesh Juneja, Congchao Wang, Zifeng Wang, Alec Go, Chen-Yu Lee, Pradeep Shenoy, Rina Panigrahy, Aditya Krishna Menon, and Sanjiv Kumar. Universal model routing for efficient llm inference. In *The Fourteenth International Conference on Learning Representations*, 2026.

[16] Kong Inc. Kong ai gateway. Product Documentation, 2026.

[17] Kubernetes Authors. Custom resource definitions. Kubernetes Documentation, 2025.

[18] Kubernetes Authors. Custom resources. Kubernetes Documentation, 2025.

[19] Kubernetes Authors. Gateway api. Kubernetes Documentation, 2025.

[20] Kubernetes Authors. Operator pattern. Kubernetes Documentation, 2025.

[21] Kyverno Authors. Kyverno introduction. Project Documentation, 2026.

[22] Kyverno Authors. Kyverno policy reports. Project Documentation, 2026.

[23] Percy Liang, Rishi Bommasani, Tony Lee, et al. Holistic evaluation of language models. *Transactions on Machine Learning Research*, 2023.

[24] Microsoft. Overview of managed identities in azure kubernetes service (AKS). Microsoft Learn, 2025.

[25] Microsoft. Azure ai foundry documentation. Microsoft Learn, 2026.

[26] Microsoft. Azure ai foundry models overview. Microsoft Learn, 2026.

[27] Microsoft. Azure monitor metrics overview. Microsoft Learn, 2026.

[28] Microsoft. Kubernetes monitoring overview. Microsoft Learn, 2026.

[29] Microsoft. Monitor agents using the dashboard. Microsoft Learn, 2026.

[30] Microsoft. Monitor azure kubernetes service (AKS). Microsoft Learn, 2026.

[31] Microsoft. Observability in azure ai foundry. Microsoft Learn, 2026.

[32] Microsoft. Secure access to azure openai from azure kubernetes service (AKS). Microsoft Learn, 2026.

[33] Microsoft. Use a microsoft entra workload id on azure kubernetes service (AKS). Microsoft Learn, 2026.

[34] Microsoft. What is azure ai foundry? Microsoft Learn, 2026.

[35] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). NIST Publication, 2023.

[36] Isaac Ong, Yilun Yao, Tianyu Zhang, Karthik Narasimhan, and He He. Routellm: Learning to route llms from preference data. In *The Thirteenth International Conference on Learning Representations*, 2025.

[37] Open Policy Agent Gatekeeper Authors. Gatekeeper documentation. Project Documentation, 2026.

[38] OpenCost Authors. Opencost overview. Project Documentation, 2026.

[39] OpenCost Authors. Opencost specification. Project Documentation, 2026.

[40] OpenTelemetry Authors. Opentelemetry overview. Specification Documentation, 2026.

[41] OpenTelemetry Authors. What is opentelemetry? Project Documentation, 2026.

[42] OWASP Foundation. Owasp top 10 for large language model applications 2025. Project Documentation, 2025.

[43] Portkey. Portkey ai gateway documentation. Product Documentation, 2026.

[44] Prometheus Authors. Prometheus data model. Project Documentation, 2026.

[45] Prometheus Authors. Prometheus overview. Project Documentation, 2026.

[46] Tetragon Authors. Tetragon enforcement. Project Documentation, 2026.

[47] Tetragon Authors. Tetragon overview. Project Documentation, 2026.

[48] Arther Tian, Alex Ding, Frank Chen, Simon Wu, and Aaron Chan. A multi-dimensional quality scoring framework for decentralized LLM inference with proof of quality, 2026.

[49] Yusuke Yamauchi, Taro Yano, and Masafumi Oyamada. An empirical study of LLM-as-a-judge: How design choices impact evaluation reliability, 2025.

[50] Li Yang et al. Let the llm stick to its strengths: Learning to route economical llm. In *OpenReview*, 2025.

[51] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *arXiv preprint arXiv:2306.05685*, 2023.

[52] Yusheng Zheng, Tong Yu, Yiwei Yang, and Andrew Quinn. Wasm-bpf: Streamlining eBPF deployment in cloud environments with WebAssembly, 2024.